

bok25

基
礎
班

、**考研数据結構**



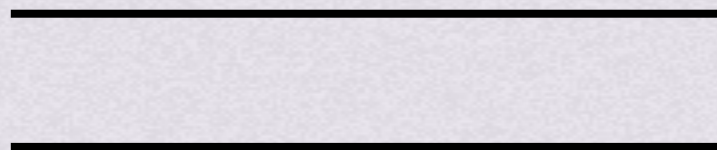
队列是什么？

先进先出的线性表

假设一组指令：入队，出队，入队，入队，入队，出队，出队，出队



队头指针



数据a

数据b

数据c

数据d

队尾指针

假设队尾指针指向队尾元素下一位置

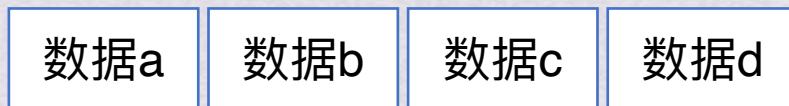


队列是什么？

先进先出的线性表

假设一组指令：入队，出队，入队，入队，入队，出队，出队，出队

最终得到的数据序列



对于同一个入队序列，如果我们规定的操作序列不同，那么出队序列也会不同。
所以，在没有限定操作序列的前提下，入队序列和出队序列会呈现一对多的关系



队列的类型定义

ADT Queue {

数据对象 : $D = \{a_i | a_i \in \text{ElemSet}, i = 1, 2, \dots, n, n \geq 0\}$

数据关系 : $R = \{ \langle a_{i-1}, a_i \rangle | a_{i-1}, a_i \in D, i = 2, \dots, n \}$

约定其中 a_1 端为队头, a_n 端为队尾。

基本操作 :

InitQueue(& Q)

操作结果 : 构造一个空队列Q。

DestroyQueue(& Q)

初始条件 : 队列Q已存在。

操作结果 : 队列Q被销毁, 不再存在。

ClearQueue(& Q)

初始条件 : 队列Q已存在。

操作结果 : 将Q清为空队列。

QueueEmpty(Q)

初始条件 : 队列Q已存在。

操作结果 : 若Q为空队列, 则返回true, 否则返回false。

QueueLength(Q)

初始条件 : 队列Q已存在。

操作结果 : 返回Q的元素个数, 即队列的长度。

GetHead(Q)

初始条件 : Q为非空队列。

操作结果 : 返回Q的队头元素。

EnQueue(& Q, e)

初始条件 : 队列Q已存在。

操作结果 : 插入元素e为Q的新的队尾元素。

DeQueue(& Q, & e)

初始条件 : Q为非空队列。

操作结果 : 删除Q的队头元素, 并用e返回其值。

QueueTraverse(Q)

初始条件 : Q已存在且非空。

操作结果 : 从队头到队尾, 依次对Q的每个数据元素进行访问。

}ADT Queue



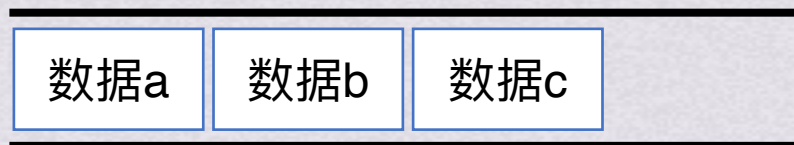
队列的顺序存储

```
//- - - - - 队列的顺序存储结构- - - - -  
#define MAXQSIZE 100 //队列可能达到的最大长度  
typedef struct  
{ QElemType *base;    //存储空间的基地址  
  int front; //头指针  
  int rear;  //尾指针  
}SqQueue;
```



队尾指针的设置

队头指针
Q.front



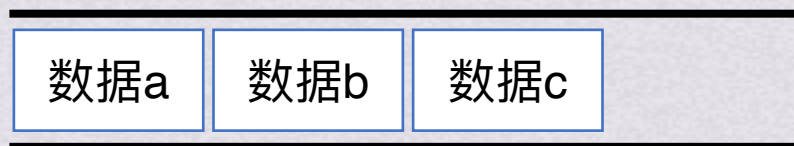
Q.rear
队尾指针





队尾指针的设置

队头指针
Q.front



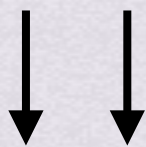
Q.rear
队尾指针



队列的缺陷

依次操作:入队, 入队, 入队, 入队, 出队, 出队

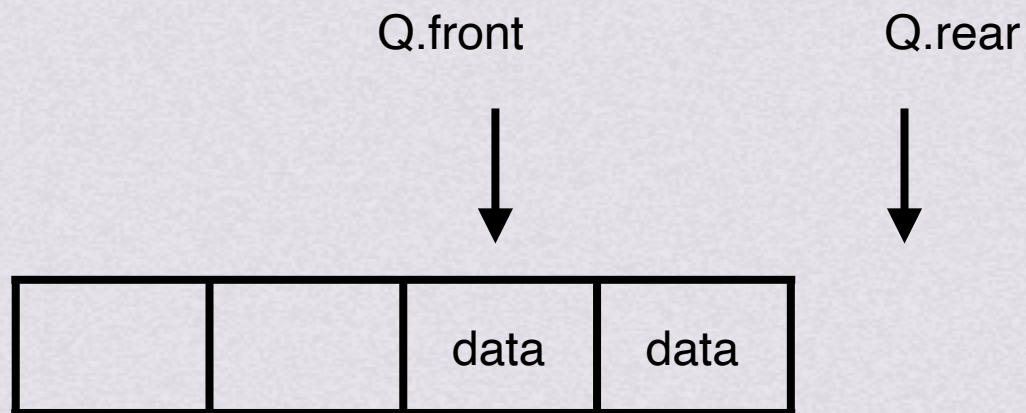
Q.front Q.rear





队列的缺陷

依次操作:入队, 入队, 入队, 入队, 出队, 出队



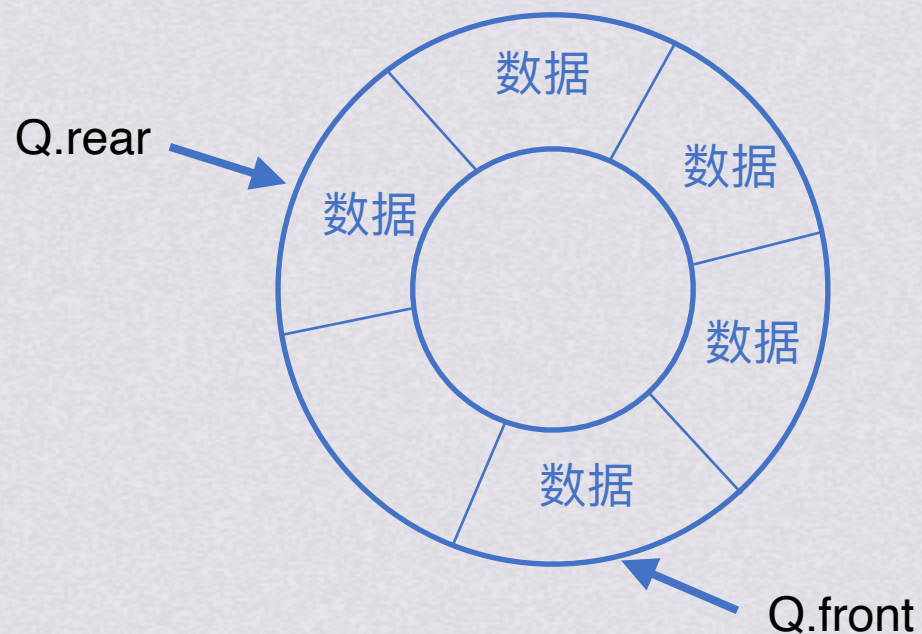
思考: 队列空吗? 还能有元素入队吗?

数组中仍存在可存放元素的空位置却无法入队的现象, 称为假溢出



队列的缺陷

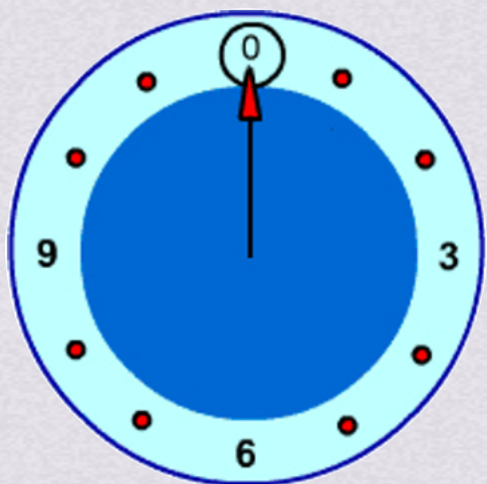
解决方法：循环队列



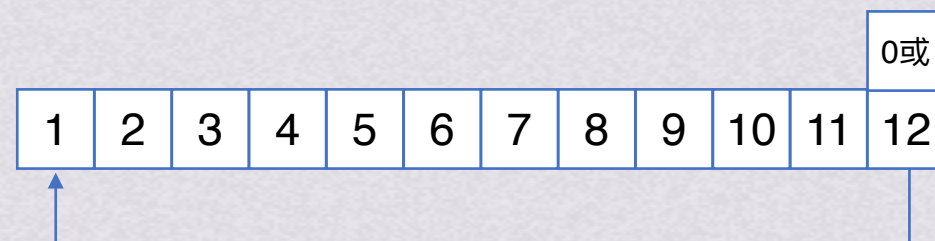


队列的缺陷

解决方法：循环队列 “循环”的实现（模的运算）



钟表：最常见的模运算



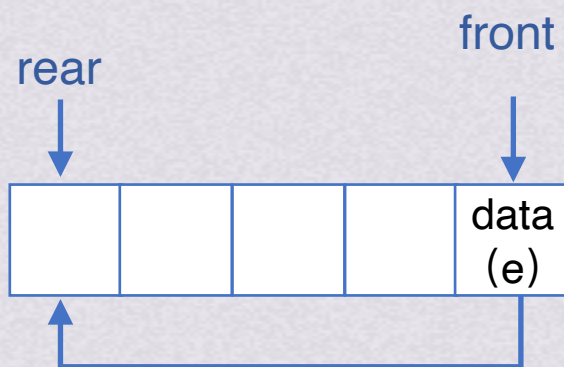
如：18点， $18\%12=6$

如：20点， $20\%12=8$



队列的缺陷解决方法：循环队列 解决假溢出实践

假设MAXQSIZE=5



此时按照循环队列的入队代码执行入队操作

假设再执行一次出队操作

e = data

循环队列的入队

```
Status EnQueue(SqQueue &Q, QElemType e)
{
    // 插入元素 e 为 Q 的新的队尾元素
    if((Q.rear+1)%MAXQSIZE==Q.front) //若尾指针在循环意义上加1后等于头指针，表明队满
        return ERROR;
    Q.base[Q.rear]=e; //新元素插入队尾
    Q.rear=(Q.rear+1)%MAXQSIZE; //队尾指针加1
    return OK;
}
```

循环队列的出队

```
Status DeQueue(SqQueue &Q, QElemType &e)
{
    // 删除 Q 的队头元素，用 e 返回其值
    if(Q.front==Q.rear) return ERROR; //队空
    e=Q.base[Q.front]; //保存队头元素
    Q.front=(Q.front+1)%MAXQSIZE; //队头指针加1
    return OK;
}
```




循环队列的初始化

```
Status InitQueue(SqQueue &Q)
{
    //构造一个空队列 Q
    Q.base=new QElemType[MAXQSIZE]; //为队列分配一个最大容量为MAXQSIZE的数组空间
    if(!Q.base) exit(OVERFLOW); //存储分配失败
    Q.front=Q.rear=0; //将头指针和尾指针置为0，队列为空
    return OK;
}
```



循环队列的初始化

```
Status InitQueue(SqQueue &Q)
{ //构造一个空队列 Q
    Q.base=new QElemType[MAXQSIZE]; //为队列分配一个最大容量为MAXQSIZE的数组空间
    if(!Q.base) exit(OVERFLOW); //存储分配失败
    Q.front=Q.rear=0; //将头指针和尾指针置为0，队列为空
    return OK;
}
```

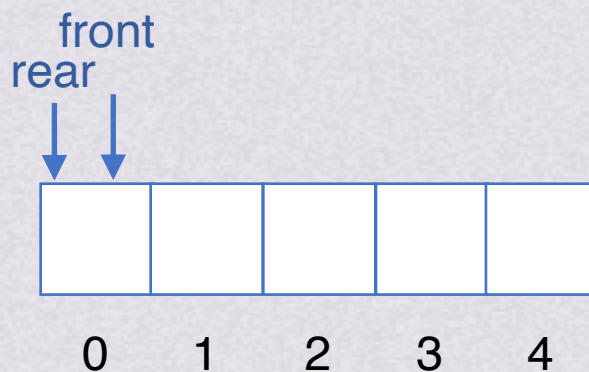
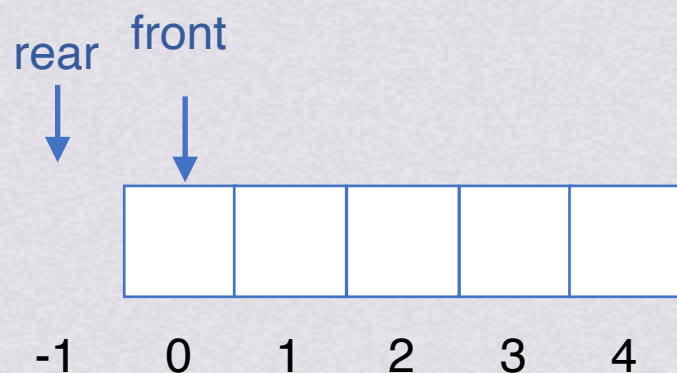
思考：如果rear改为指向队尾元素，初始化栈空时该如何表达？

答案：Q.front=0，Q.rear=-1。

【此答案仅供理解，Q.rear=-1仅仅为了表示Q.rear在Q.front之前，实际数组没有-1为下标的空间，代码会报错】



顺序队列不同rear指针下情况小结



1. 设front指向队头，rear指向队尾

```
EnQueue{  
    //入队  
    rear++;  
    Q.data[rear]=x;}  
  
DeQueue{  
    //出队  
    x=Q.data[front];  
    front++;}
```

2. 设front指向队头，rear指向队尾后一个元素

```
EnQueue{  
    //入队  
    Q.data[rear]=x;}  
    rear++;  
  
DeQueue{  
    //出队  
    x=Q.data[front];  
    front++;}
```



循环队列求长度

```
int QueueLength(SqQueue Q)
{ // 返回 Q 的元素个数，即队列的长度
    return (Q.rear - Q.front + MAXQSIZE) % MAXQSIZE;
}
```

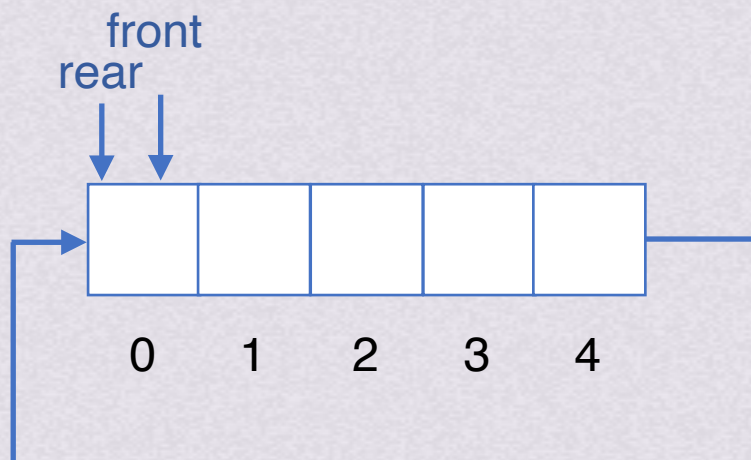



取队头元素

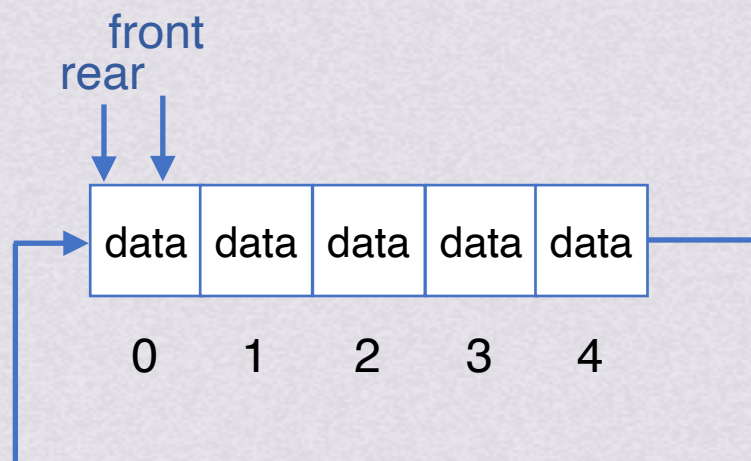
```
QElemType GetHead(SqQueue Q)
{ // 返回 Q 的队头元素，不修改队头指针
    if(Q.front != Q.rear) // 队列非空
        return Q.base[Q.front]; // 返回队头元素的值，队头指针不变
}
```



循环队列新的缺陷



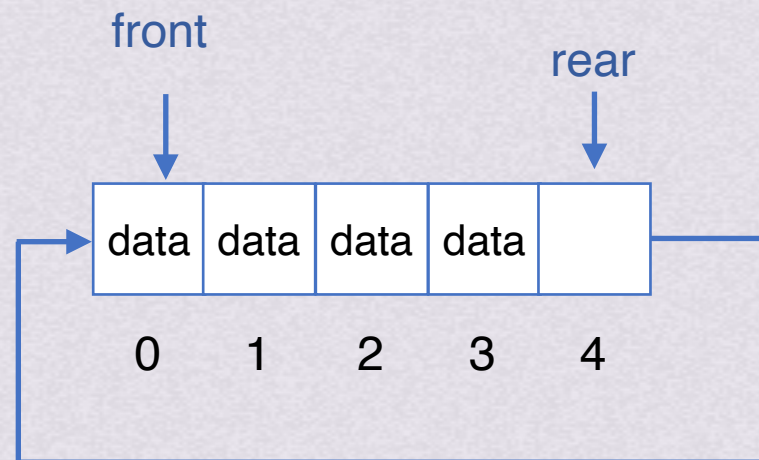
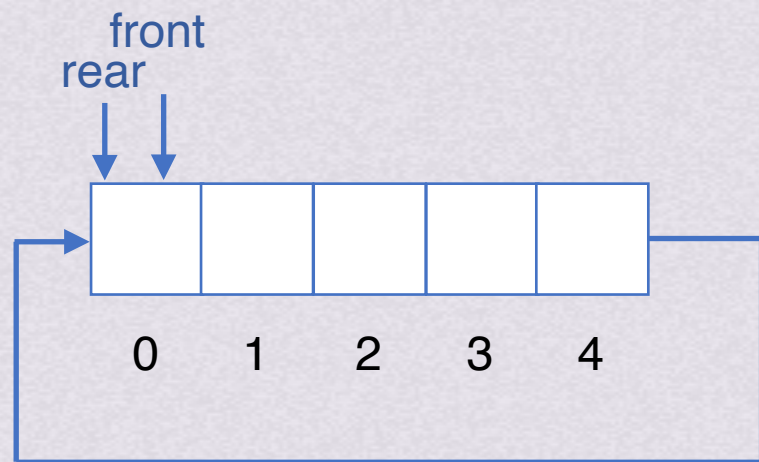
队空: $\text{front} == \text{rear}$



队满: $\text{front} == \text{rear}$



循环队列新的缺陷 解决方案



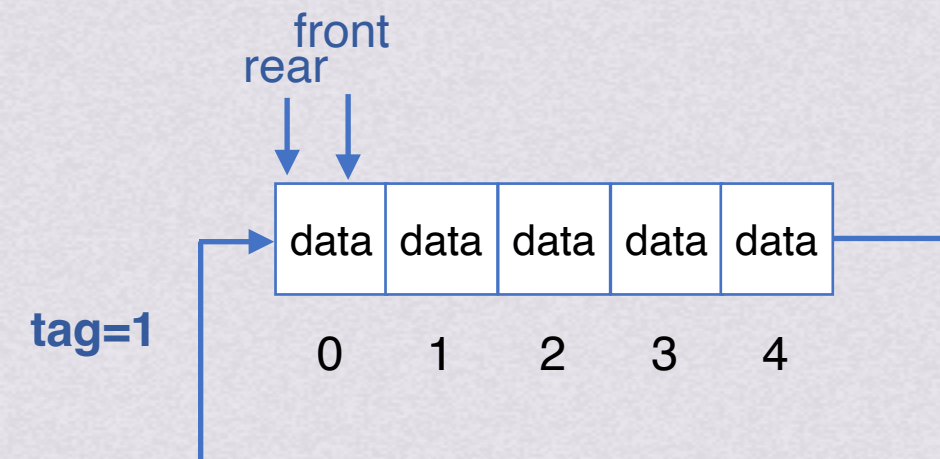
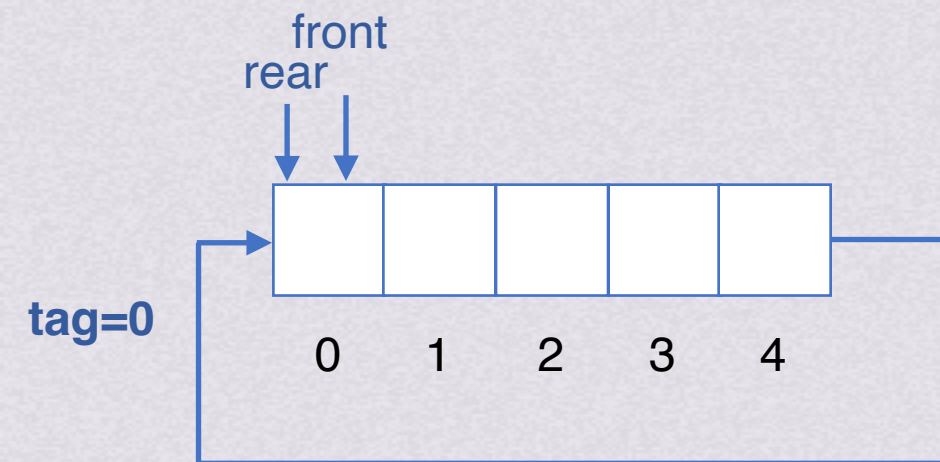
方法一：少用一个元素空间，即当队列空间大小为 m 时，有 $m-1$ 个元素就认为是队满

队空的条件： $Q.front == Q.rear$

队满的条件： $(Q.rear + 1) \% MAXQSIZE == Q.front$



循环队列新的缺陷 解决方案



方法二：另设一个标志位以区别队列是“空”还是“满”

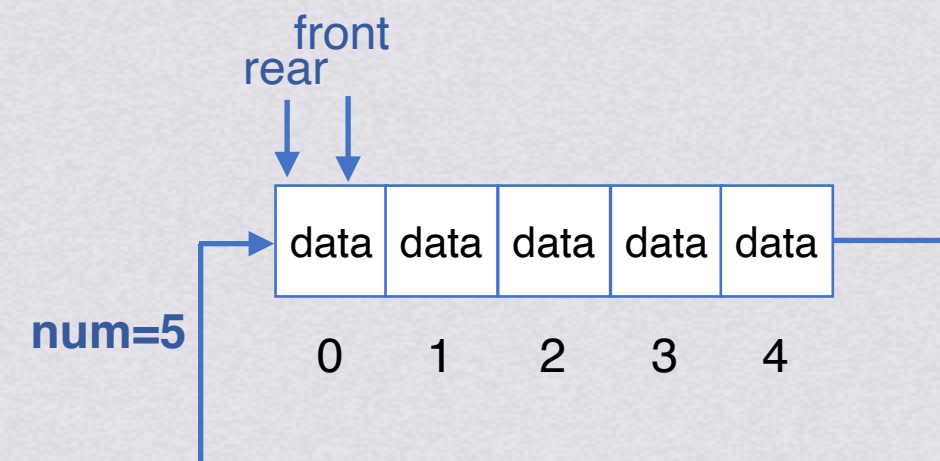
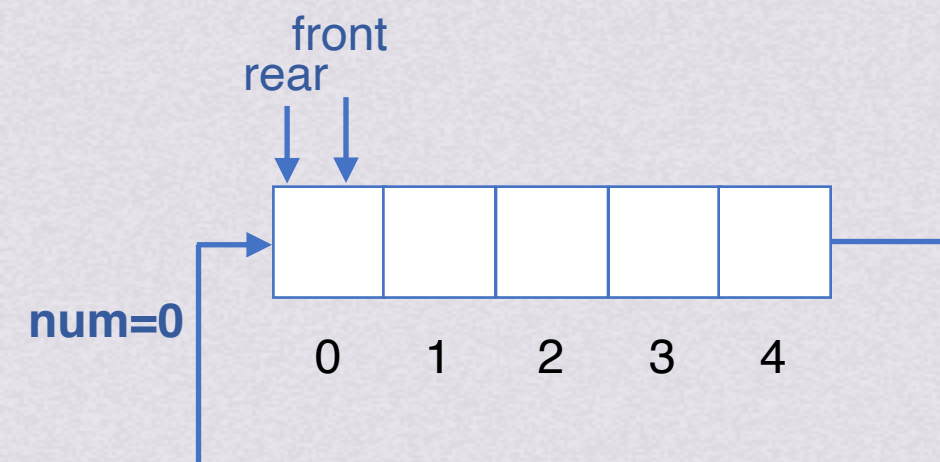
增设一tag位区分队满或队空，
删除成功置 $tag=0$ ，插入成功置 $tag=1$

$tag=0$ 时，因删除导致的 $Q.front == Q.rear$ 为队空；

$tag=1$ 时，因插入导致 $Q.front == Q.rear$ 为队满



循环队列新的缺陷 解决方案

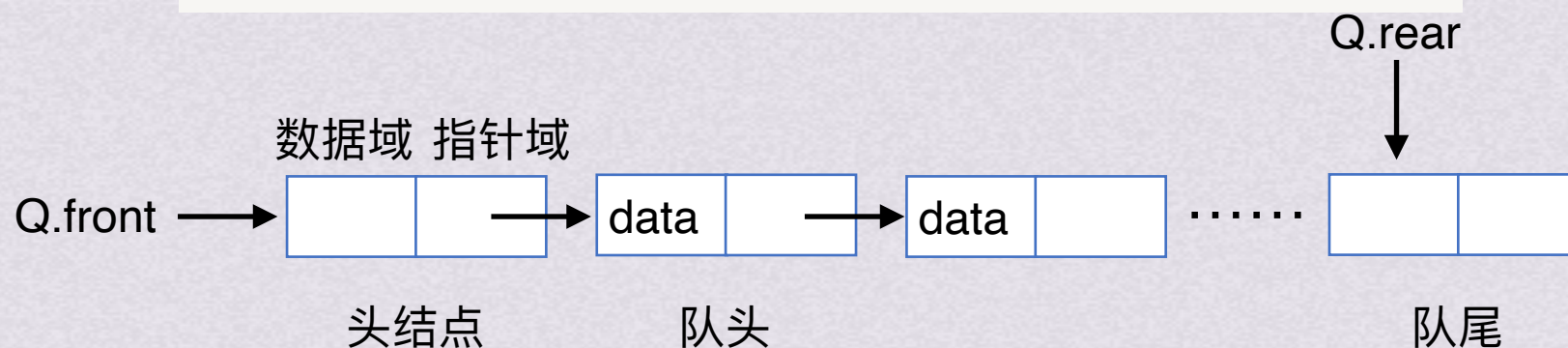


方法三：设置num变量记录队列中元素个数



链队列

```
//- - - - - 队列的链式存储结构- - - - -  
typedef struct QNode{  
    QElemType data;  
    struct QNode *next;  
}QNode, *QueuePtr;  
typedef struct{  
    QueuePtr front;    //队头指针  
    QueuePtr rear;     //队尾指针  
}LinkQueue;
```





链队列的初始化

```
Status InitQueue(LinkQueue &Q)
{ // 构造一个空队列 Q
    Q.front=Q.rear=new QNode; //生成新节点作为头节点，队头和队尾指针指向此节点
    Q.front->next=NULL; //头节点的指针域置空
    return OK; }
```

注：QNode和*QueuePtr表示的是同一个结构体，只不过QNode是结构体类型，*QueuePtr是指向这个结构体的指针类型

$(*a).b = a \rightarrow b$



链队列的入队

无需判断是否队满，直接动态分配空间即可

```
Status EnQueue(LinkQueue &Q, QElemType e){  
    // 插入元素 e 为 Q 的新的队尾元素  
    p=new QNode; //为入队元素分配节点空间，用指针p指向  
    p->data=e;    //将新节点数据域置为e  
    p->next=NULL; Q.rear->next=p; //将新节点插入队尾  
    Q.rear=p;      // 修改队尾指针  
    return OK;  
}
```




链队列的出队

```
Status DeQueue(LinkQueue &Q, QElemType &e)
{
    // 删除 Q 的队头元素，用 e 返回其值
    if(Q.front==Q.rear) return ERROR; //若队列为空，则返回ERROR
    p=Q.front->next; //p指向队头元素
    e=p->data; //e保存队头元素的值
    Q.front->next=p->next; //修改头节点的指针域
    if(Q.rear==p) Q.rear=Q.front; //最后一个元素被删，队尾指针指向头节点
    delete p; //释放原队头元素的空间
    return OK;
}
```



取队头元素

```
QElemType GetHead(LinkQueue Q)
{
    // 返回 Q 的队头元素，不修改队头指针
    if(Q.front != Q.rear) // 队列非空
        return Q.front->next->data; // 返回队头元素的值，队头指针不变
}
```